Agent Control Plane Reference Architecture (ACP-RA)

Governed, scalable agentic autonomy for contested and degraded operations

Adam Boas

Agentic Warfare Architect, Department of War

February 2026

Contents

Executive Summary

Agentic autonomy is shifting the unit of output from *human-hours* to *agent-hours*. That is the transition from "force multiplication" to "force creation": intent-driven systems executing within delegated authority, scaling through silicon rather than staffing. In practical terms, decision tempo and execution density can exceed human relevance windows—especially in contested environments where connectivity is intermittent and deception is routine.

The Agent Control Plane (ACP) exists to make this transition **fieldable**.

It is the mechanism that lets the Department move faster **without** turning speed into unmanaged risk. It does this by standardizing and enforcing:

- **Identity** for agents as non-person entities (NPEs)
- **Delegated authority** as explicit trust scopes
- **Work units** as the supervised unit of long-running, parallel agent execution
- **Mediated action** through tool and inter-agent gateways
- **Governance as code** with continuous evaluation and promotion gates
- **Evidence and replay** sufficient for continuous authorization and after-action reconstruction
- **Swarm/ensemble governance** so multi-agent coordination remains bounded, attributable, and containable
- **Degraded-mode survivability** so autonomy degrades safely when networks, models, or services are denied

This document is written as a DoW CIO–style reference architecture: strategic purpose, principles, technical positions, patterns, and vocabulary. It is intended to guide and constrain downstream solution architectures rather than prescribe a single implementation.

1 Agent Control Plane Reference Architecture (ACP-RA)

This section defines ACP-RA at a glance: what it covers, why it exists, and the baseline principles it enforces. The goal is to establish a shared frame before diving into specific control surfaces and patterns.

1.1 Executive summary

Agentic autonomy is shifting the unit of output from *human-hours* to *agent-hours*. That is the transition from "force multiplication" to "force creation": intent-driven systems executing within delegated authority, scaling through silicon rather than staffing. In practical terms, decision tempo and execution density can exceed human relevance windows—especially in contested environments where connectivity is intermittent and deception is routine.

The Agent Control Plane (ACP) exists to make this transition **fieldable**.

It is the mechanism that lets the Department move faster **without** turning speed into unmanaged risk. It does this by standardizing and enforcing:

- **Identity** for agents as non-person entities (NPEs)
- **Delegated authority** as explicit trust scopes

- **Work units** as the supervised unit of long-running and parallel agent execution
- **Mediated action** through tool and inter-agent gateways
- **Governance as code** with continuous evaluation and promotion gates
- **Evidence and replay** sufficient for continuous authorization and after-action reconstruction
- **Swarm/ensemble governance** so multi-agent coordination remains bounded, attributable, and containable
- **Degraded-mode survivability** so autonomy degrades safely when networks, models, or services are denied

This document is written as a DoW CIO–style reference architecture: strategic purpose, principles, technical positions, patterns, and vocabulary. It is intended to guide and constrain downstream solution architectures rather than prescribe a single implementation.

1.2 Scope and non-goals

Before debating design details, we bound the problem. ACP-RA focuses on the control-plane mechanisms that make agent execution governable at scale. It does not attempt to prescribe mission tactics, weapon employment, or policy beyond the control plane.

1.2.1 In scope

- Enterprise, intelligence, and operational-support agents that plan, coordinate, and invoke tools/actions within bounded authority
- Multi-agent ensembles (swarms) and their coordination, messaging, shared state, budgets, observability, and containment
- Work-unit management for long-running and parallel agent execution (checkpointing, dependencies, cancellation, supervision)
- Policy-as-code, evaluation-as-gate, and evidence generation for continuous authorization
- Cross-enclave federation and cross-domain handoffs (identity, context, and evidence)

1.2.2 Out of scope

- Tactical employment guidance for autonomous weapons
- Authorization of use-of-force decisions
- Any design that bypasses applicable weapon system autonomy policy (see DoDD 3000.09)

Where agents integrate into systems adjacent to use-of-force or mission-critical safety, additional governance, testing, and policy applies. DoDD 3000.09 remains the governing policy for autonomy in weapon systems.
(References: DoDD 3000.09; see Appendix “References.”)

1.3 Strategic drivers

The drivers below describe why the Department needs an ACP now. They are constraints that shape requirements: speed without loss of governance, supervision at scale, resilience under denial, and interoperability across federated enclaves.

1.3.1 A posture of acceleration

Recent Department strategy and senior-leadership messaging emphasize acceleration in AI adoption, experimentation, compute access, and rapid iteration. This architecture treats that posture as a constraint: platforms must support rapid onboarding and change while staying governable and reversible.

1.3.2 The core problem is shifting from "can agents do X?" to "can people supervise agents at scale?"

Commercial agent systems are converging on a command-center model: many tasks in parallel, long-running execution, and human supervision as a portfolio function rather than a per-action bottleneck. For the Department, this shift is not cosmetic—it drives requirements for work-unit identity, evidence indexing, pause/resume semantics, and escalation controls that remain enforceable at machine tempo.

1.3.3 Contested and degraded environments are the baseline, not the exception

Agentic systems fail differently than traditional software. They are vulnerable to deception, poisoning, and cascade failures—especially when they coordinate as ensembles. The control plane must survive partial connectivity, intermittent access to centralized services, and adversary attempts to subvert policy and evidence mechanisms themselves.

1.3.4 Interoperability and federation are unavoidable

DoW reality is federated: Services, agencies, mission partners, and multiple classification enclaves. Interoperability is also plural: agent-to-agent messaging and agent-to-tool/data connectors are both being standardized in industry. The ACP must be **protocol-neutral** while enforcing a consistent policy surface across whichever interop protocols are in use.

1.4 Architectural principles

1.4.1 P1 — Agents are non-person entities (NPEs), not "apps"

Agents must have identity, credentials, lifecycle controls, and attributes consistent with enterprise identity patterns. Treat them as first-class actors with accountability.

1.4.2 P2 — Centralize policy decisions; distribute enforcement

The ACP uses a policy decision point (PDP) with multiple policy enforcement points (PEPs): at runtime admission, model routing, context retrieval, tool invocation, inter-agent messaging, and work-unit state transitions.

1.4.3 P3 — Default deny; allow by explicit trust scope

Agents do not gain authority because a prompt implies it. Authority is granted by a signed, versioned trust scope manifest and enforced by gateways.

1.4.4 P4 — The "doing boundary" is explicit

Tool calls and other side effects are mediated. Every action is a policy event. "It can think" is not the same as "it can do."

1.4.5 P5 — Evidence is first-class

Evidence is produced at the same tempo as actions. Continuous authorization and credible after-action review require deterministic, replayable artifacts.

1.4.6 P6 — Rollback and containment are capabilities

Speed wins only if rollback is instant and scoped. Containment must isolate a single agent without collapsing an ensemble.

1.4.7 P7 — Context engineering is governed data movement

Context is not a prompt trick; it is a data plane. Provenance, minimization, freshness, and labeling are mandatory.

1.4.8 P8 — Evals and monitoring are gates

Continuous evaluation, red-teaming, drift detection, and anomaly response are not documentation—they are release and runtime gates.

1.4.9 P9 — Budgets are policy, not accounting

Compute, bandwidth, tool calls, and power are operational constraints. Autonomy must operate within explicit budgets—especially in contested logistics.

1.4.10 P10 — Tools/skills are a supply chain

Agent capability expansion via tools, connectors, and "skills" is unavoidable. Tool ecosystems become attack surfaces. Therefore: tools are onboarded, signed, scanned, evaluated, and attested like software packages; execution is sandboxed and governed by trust scope.

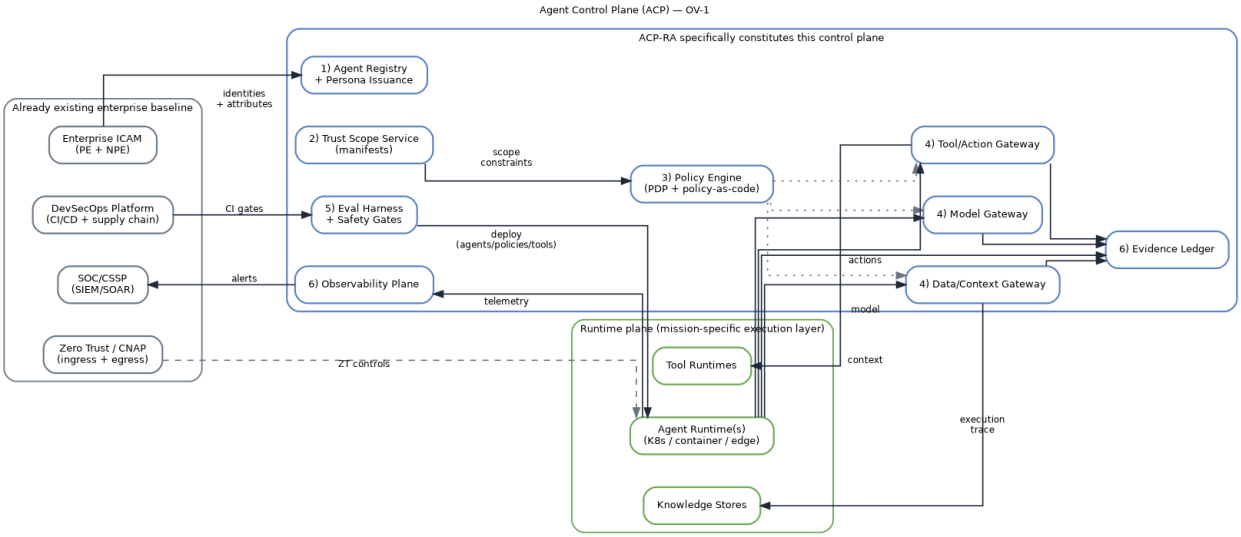


Figure 1: ACP OV-1

1.5 Reference architecture structure

DoW CIO reference architectures guide and constrain downstream architectures by providing: **strategic purpose, principles, technical positions, patterns, and vocabulary**. ACP-RA uses that same structure.

- **Strategic purpose:** why ACP exists
- **Principles:** non-negotiable engineering behavior
- **Technical positions:** required control surfaces and boundary points
- **Patterns:** reusable implementation guidance
- **Vocabulary:** consistent terms that prevent "same word, different meaning" failures

---

1.6 Conceptual view (OV-1)

The conceptual model is a control system with two planes:

- **Runtime plane:** agent runtimes executing plans and invoking tools (K8s, edge nodes, mission systems)
- **Control plane:** identity, trust scopes, work units, policy, gateways, evaluation, evidence, observability, and containment

The ACP does not "run the mission." It constrains, mediates, and proves mission execution.

---

2 Core vocabulary

This section defines primitives that remain stable even as frameworks change.

2.1 Agent

A software actor that can plan, retrieve context, communicate, invoke tools, and execute actions toward a goal within bounded authority.

2.2 Persona

A controlled mission role for an agent (e.g., enterprise drafting, logistics planner, intel triage, policy checker). Persona is an attribute used by policy.

2.3 Principal

A security subject that can hold permissions and be audited. A principal may be a human identity (ICAM subject) or a non-person entity (NPE) such as an agent runtime, tool runtime, or service.

2.4 Delegation chain

A signed binding that captures the principals involved in an execution step:

- `ownerPrincipal`: the human or organization principal that created/owns the agent configuration.
- `agentServicePrincipal`: the service principal under which the agent executor runs.
- `onBehalfOfPrincipal`: the human principal whose authority is being exercised at a particular point in time (optional; required for delegated-user semantics).

Delegation does not create authority. It selects whose authority is being exercised under the constraints of the trust scope and policy bundle. Effective authority composes by intersection across the trust scope and all principals present in the chain.

2.5 Trust boundaries: authority sources vs data sources

2.5.1 Authority sources (may authorize actions)

1. Authenticated human principals and authenticated service principals.
2. Signed trust scope manifests and signed policy bundles valid for the target environment.
3. Policy decisions produced by authorized PDP/PEP enforcement at runtime.

Delegation context is an authority *input*, not an authority *source*. Prompts, documents, or tool outputs MUST NOT be allowed to assert "on behalf of" execution. If an agent is acting on behalf of a human principal, that binding MUST be derived from an authenticated session and recorded as evidence; otherwise, the agent operates only as its own NPE identity under its trust scope.

Declared intent is not an authority source. It is a claim recorded for accountability and policy evaluation; authorization still derives from authenticated principals, trust scope constraints, and policy decisions.

2.5.2 Data sources (inform decisions, never grant authority)

- Prompts, retrieved documents, tool outputs, and model-generated text are data inputs only.
- Data can influence prioritization and recommendation, but cannot grant privileges or delegation rights.

2.6 Trust Scope Manifest (TSM)

A signed, versioned contract defining an agent's delegated authority and constraints:

- allowed/prohibited actions and tools
- consequence tier
- operating environment (enclave/classification/connectivity)
- uncertainty thresholds and escalation triggers
- budgets (compute/tokens/tool calls/egress/power/time)
- evidence requirements (fields, redactions, retention)
- approved model profiles (MAP allowlist) and model usage modes by tier
- policy trust anchors: accepted signing authorities (keys/certs) and allowlisted policy bundle and trust scope hashes per environment
- degraded-mode behaviors and containment semantics

2.7 Work unit

A durable, supervised task thread for long-running and parallel agent execution.

A work unit binds:

- a **scope context** (trust scope hash + policy bundle hash)
- **budgets** (initial allocation + delegated allowances)
- **dependencies** (work-unit DAG and blocking conditions)
- **checkpoint policy** (what is persisted, when, and how to resume)
- **sandbox provenance** (execution environment identifiers / attestations where available)
- **evidence root** (represented as `evidenceRootHash`; the stable anchor used to query all actions/messages/artifacts for this work unit, anchored in the Evidence Ledger)

Work units are how the Department supervises autonomy at scale: humans do not "watch every tool call," they supervise work units, review diffs, and intervene on escalation triggers.

2.8 Policy bundle

A signed, versioned set of policy rules consumed by enforcement points:

- ABAC rules
- escalation rules
- budget policies
- inter-agent messaging policies
- safety interlocks (quarantine/kill/rollback)

- work-unit transition constraints (pause/resume/cancel)

Policy authoring methods vary (boards, working groups, automated review pipelines, or hybrid workflows). ACP is agnostic to how policy is deliberated; it is strict about how policy becomes enforceable. Only signed, versioned policy bundles and trust scopes grant authority at runtime. Deliberation transcripts, discussions, and draft "constitutions" are procedural artifacts and MAY be recorded as evidence or procedural memory, but they MUST NOT be treated as executable authorization.

2.9 Model assurance profile (MAP)

A signed, versioned artifact that describes the assurances and constraints of a model endpoint. A MAP is referenced by immutable hash and used by the model gateway for routing and enforcement.

A MAP SHOULD include:

- model identifier and version (or immutable build hash)
- hosting boundary (enclave/region) and allowed deployment environments
- data handling constraints (prompt/completion retention, training use, telemetry content)
- security posture requirements (encryption, network boundary, authentication)
- approved usage modes (interactive assistance vs autonomous agent execution)
- eval coverage references (required eval packs, red-team suites, canary criteria)
- rollback and deprecation signals (upgrade windows, kill-switch semantics)

2.10 Evidence export profile (EEP)

A signed, versioned configuration that defines how canonical ACP evidence is transformed into derived event records for external consumers (for example SIEM pipelines or analytics systems). EEPs do not change what ACP records; they standardize how evidence is represented for downstream systems. If an implementation exports derived evidence, it MUST do so via an EEP; otherwise EEPs are not required.

An EEP MUST define:

- target schema and profile identifier (when applicable)
- required linkage fields (`envelopeHash`, `evidenceRootHash`, `workUnitId`, `trustScopeRef`, `policyBundleRef`)
- field-level minimization and redaction rules
- label propagation and access constraints for exported records
- export destinations as governed sinks (export is an action, not a background task)

2.11 Context bundle

A replayable artifact representing retrieved/used context:

- source pointers, timestamps, labels/tags
- provenance and integrity metadata
- minimization decisions
- freshness/rot signals
- a stable hash identifier (so later evidence can reference it)

2.12 Action envelope (minimum required fields + extension blocks)

A signed record of an attempted tool/action. Example (minimum required fields + optional extension blocks):

The "Minimum required properties" list defines what is mandatory; any other blocks shown in the example are optional extensions unless required by policy/trust scope.

```
actionEnvelope:
  envelopeVersion: "v1"
  actionId: "act-..."
  workUnitId: "wu-..."
  actor:
    principalRef: "principal://npe/service/..."
    persona: "ops-planner"
  delegation:
    ownerPrincipal: "principal://icam/subject/..."
    agentServicePrincipal: "principal://npe/service/..."
    onBehalfOfPrincipal: "principal://icam/subject/..."     # optional; required for
↪  delegated-user semantics
    sessionRef: "session://sha256:..."                      # binds onBehalfOfPrincipal to an
↪  authenticated session
    delegationHash: "sha256:..."                            # canonical hash of delegation
↪  fields excluding delegationHash
  intent:
    declaredPurpose: "Create a pull request to remediate CVE-2026-XXXX in repository X"
    expectedEffects:
      - "modify:repo://X/path/to/file"
      - "create:repo://X/pull-request"
    consequenceTier: "t2"
    intentHash: "sha256:..."                                # canonical hash of intent fields
↪  excluding intentHash
  scope:
    trustScopeRef: "trustscope://...@sha256:..."
    policyBundleRef: "policy://...@sha256:..."
  context:
    bundleRefs:
      - "context://...@sha256:..."
  lineage:
    model:
      modelId: "model:..."
      modelVersion: "sha256:..."
      mapRef: "map://sha256:..."
    tool:
      toolId: "tool:github.pull_request.create"
      toolVersion: "1.3.2"
    outputs:
      - artifactHash: "sha256:..."
        artifactRef: "evidence://artifact/sha256:..."
        labels: ["artifact:pr", "tier:t1"]
  evidence:
    envelopeHash: "sha256:..."
    evidenceRootHash: "sha256:..."
    traceId: "trace://..."
  transport:                                                # optional; only for
↪  tool-invocation actions
```

```
    protocol: "<string>"                                    # e.g., "mcp", "grpc", "https"
    serverId: "tool-server://..."
    clientId: "agent-client://..."
    connectionMode: "<string>"                              # e.g., "stdio", "http", "sse"
  decision:
    verdict: "allow"
    decisionRef: "decision://..."
  outcome:
    status: "success"
    completedAt: "2026-02-16T00:00:00Z"
  signature:
    signer: "principal://npe/service/..."
    signatureAlgorithm: "ed25519"
    signatureValue: "base64:..."
```

Minimum required properties (non-exhaustive):

- `workUnitId`, `trustScopeRef`, `policyBundleRef`, and policy decision references.
- Delegation binding: `delegationHash` and signature-covered delegation fields; `onBehalfOfPrincipal` MUST be tied to an authenticated `sessionRef` when present.
- Intent is an evidence-bearing claim. It is OPTIONAL for low-consequence reversible operations, but MUST be present (with `intentHash`) for irreversible actions and/or consequence tiers T2+ (as defined by the policy bundle / trust scope).
- Lineage binding: `toolId`/`toolVersion`, `modelId`/`modelVersion` (or build hash) when applicable, and produced artifact hashes MUST be present and signature-covered to support trace-to-version replay.

Additional normative rules:

- `delegation` is OPTIONAL.
- If `onBehalfOfPrincipal` is present, `sessionRef` MUST be present.
- `delegationHash` MUST be computed over canonicalized delegation fields and covered by the envelope signature (not agent-supplied).
- `intentHash` MUST be computed over canonicalized intent fields and MUST be signature-covered in the action envelope.
- `envelopeHash` and `evidenceRootHash` MUST be computed by the gateway/evidence-ledger writer (not supplied by the agent).
- `traceId` MUST be generated by the platform/gateway and propagated consistently across exports and observability.

Canonicalization and hash computation rules: Hashes MUST be computed over a deterministic canonical serialization of the input object (e.g., RFC 8785 JSON Canonicalization Scheme). `delegationHash` MUST be computed over the `delegation` object excluding the `delegationHash` field itself. `intentHash` MUST be computed over the `intent` object excluding the `intentHash` field itself. `envelopeHash` MUST be computed over the entire envelope excluding the `signature` block and excluding `evidence.envelopeHash` (to avoid self-reference). `evidenceRootHash` MUST refer

to the work unit's evidence-root anchor maintained by the Evidence Ledger (keyed by `workUnitId`), not a per-envelope Merkle root.

2.13 Inter-agent message envelope (minimum schema)

A signed record of a message between agents (or ensembles). Minimum schema example:

```yaml
messageEnvelope:
  envelopeVersion: "v1"
  messageId: "msg-..."
  sender:
    principalRef: "principal://npe/service/..."
    persona: "ops-planner"
  recipients:
    - "principal://npe/service/..."
  sequenceNumber: 14
  ttlSeconds: 120
  payloadHash: "sha256:..."
  scope:
    workUnitId: "wu-..."
    trustScopeRef: "trustscope://...@sha256:..."
    policyBundleRef: "policy://...@sha256:..."
    delegation:
      ownerPrincipal: "principal://icam/subject/..."
      agentServicePrincipal: "principal://npe/service/..."
      onBehalfOfPrincipal: "principal://icam/subject/..."     # optional
      sessionRef: "session://sha256:..."
      delegationHash: "sha256:..."
  signature:
    signer: "principal://npe/service/..."
    signatureAlgorithm: "ed25519"
    signatureValue: "base64:..."
```

Replay protection and signature coverage:

- Signature coverage: the sender signature MUST cover recipients, sequenceNumber, TTL, payloadHash, workUnitId, trustScopeRef, policyBundleRef, and delegationHash (when present).

2.14 Ensemble (swarm)

A first-class multi-agent object with explicit governance:

- membership and roles
- coordination pattern and arbitration rules
- shared-state rules and boundaries
- ensemble budgets and allocation policy
- inter-agent communication policy
- containment semantics (isolate member vs freeze ensemble)

3 Consequence tiers and required controls

Trust scopes and ensembles reference consequence tiers explicitly. Consequence is the most effective way to translate "mission impact" into enforceable controls.

Tier	Representative activities (non-exhaustive)	Minimum required controls
T0 — Low	drafting, summarization, analysis with read-only context	NPE identity; data/tag controls; basic evidence logging; deny-by-default tool use
T1 — Moderate	PR creation, ticket automation, config recommendations (no direct apply)	tool mediation; budgets (tokens/tool calls/egress); baseline eval pack; drift monitoring
T2 — High	executing changes in production, account/permission changes, external comms; GUI-based operations in controlled environments	human-on-the-loop approvals; dual-control for sensitive actions; enhanced evidence; rollback rehearsal
T3 — Life/Safety / Mission-Critical	actions affecting safety, critical infrastructure, or kinetic-adjacent effects	strict deny-by-default; quorum approvals; constrained degraded modes; full-fidelity evidence on trigger; mandatory after-action review

Composition rule: when scopes combine (agent→agent, agent→ensemble, cross-enclave), authority composes by **intersection**, not union. The effective authority is the overlap of allowed actions, data, and environments—never the sum.

---

4 Technical positions (required control surfaces)

ACP-RA constrains solution architectures through required control surfaces and boundary points.

4.1 TP1 — Agent identity as NPE (ICAM-aligned)

Every agent and tool-runtime has an identity, attributes, and lifecycle controls. Sponsorship and ownership are attributable.

4.2 TP2 — Trust scopes are signed, versioned, and enforceable

Trust scopes are artifacts. Enforcement points validate and cache them. Scope changes require promotion gates.

4.3 TP3 — Work units are first-class governance objects

Every long-running agent effort is a work unit with:

- bounded scope and budgets
- explicit dependencies and cancellation semantics
- checkpoint and resume behavior
- evidence anchoring for replay

4.4 TP4 — Tool/Action Gateway mediates all "doing"

No direct access from model runtime to privileged tools. Tool calls are policy checked, budgeted, sandboxed, and logged.

4.5 TP5 — Tools/skills are onboarded through supply-chain controls

Tools, connectors, and skills are:

- registered
- signed
- scanned (static + dependency + behavior checks)
- evaluated with tool-specific eval packs
- attested at runtime (hashes, signatures, scanner verdicts)

4.6 TP6 — Inter-Agent Gateway governs agent-to-agent communication

Agent-to-agent messaging is authenticated, authorized, schema-validated, rate-limited, and attributable. It has circuit breakers to prevent cascades.

4.7 TP7 — Context/Data Gateway governs context engineering

Context retrieval is policy checked, tag-aware, provenance-preserving, and freshness-aware. Context becomes a replayable bundle.

4.8 TP8 — Model Gateway governs model routing and upgrades

Model usage is controlled by allowlists, routing policies, canary/rollback, and evidence capture. The ACP never assumes a single model or vendor.

4.9 TP9 — Evaluation harness gates promotion; monitoring gates runtime

Evals are blocking checks (functional + policy conformance + adversarial tests). Runtime monitors detect drift and trigger containment.

4.10 TP10 — Evidence ledger is tamper-evident and queryable

Actions, context bundles, policy decisions, approvals, inter-agent envelopes, and containment events generate structured evidence sufficient for replay and continuous authorization.

4.11 TP11 — Degraded-mode behaviors are declared and enforced

When connectivity, model access, or centralized policy is denied, the system transitions to a defined degraded mode with tightened authority.

---

5 Architecture components

The ACP is best understood as a set of "bulkheads" and "gates." Bulkheads contain failures. Gates enforce policy.

5.1 Agent registry and persona issuance

This component issues and manages agent identities and personas:

- NPE identifiers and credentials
- ownership/sponsorship binding
- attribute issuance for ABAC (mission, enclave, tier, approved tools)
- lifecycle: onboarding, rotation, revocation

5.2 Trust scope service

The trust scope service stores and signs manifests, enforces schema, and supports delegation:

- scope templates by tier and mission thread
- scope inheritance and delegation (budgets, tool subsets)
- scope translation and intersection rules for federation

5.3 Work Unit Service (WUS)

This service creates and tracks work units as supervised objects:

- issues work_unit_id and binds it to trust scope + policy bundle hashes
- tracks work-unit state (queued/running/paused/blocked/canceled/completed)
- tracks dependencies (work-unit DAG) and deadlock timeouts
- manages checkpointing and resume policies (including degraded modes)
- allocates and reclaims budgets (and delegated allowances) for sub-agents
- provides a stable evidence anchor for querying actions/messages/artifacts at scale

5.4 Supervision console (Human Direction and Oversight Surface)

This is not "a UI." It is an operational control surface that enables humans to supervise autonomy at scale without becoming the throughput bottleneck.

Minimum capabilities:

- work-unit dashboard (status, dependencies, budget burn-down, anomaly flags)
- review surfaces for outputs (diff review for code/config; artifact review for plans)
- approval queue (human-on-the-loop and quorum workflows)

- intervention controls (pause/resume/cancel; quarantine/kill; tighten scope)
- evidence drill-down by work_unit_id (macro→meso→micro replay tiers)
- after-action review workflow that generates new eval cases and policy refinements

5.5 Policy engine (PDP) and distributed enforcement (PEPs)

The policy engine evaluates requests using:

- agent identity attributes + persona
- trust scope claims
- work-unit state and constraints
- environment signals (enclave, connectivity mode, posture)
- resource tags (data labels, tool categories)
- delegation context (`ownerPrincipal`, `agentServicePrincipal`, and `onBehalfOfPrincipal` where applicable), bound to an authenticated session claim
- declared intent (`intentHash` and structured intent fields), treated as an evidence-bearing claim and validated against trust scope, policy, and observed actions
- current budgets and risk posture

PEPs exist at:

- runtime admission control
- tool/action gateway
- context/data gateway
- inter-agent gateway
- model gateway
- work-unit transitions (pause/resume/cancel)
- CI/CD promotion gates

5.6 Runtime admission control (executor substrate)

Agent orchestration depends on a compute substrate that is resilient, observable, and insulated. ACP treats executor placement as a policy-enforced admission problem, not an implementation detail.

Runtime admission control enforces:

- identity and scope validity: agent identity, persona, trust scope hash, and policy bundle hash MUST be validated before an executor can run.
- delegated execution validity: if `onBehalfOfPrincipal` is asserted, it MUST be bound to a current authenticated `sessionRef` and included in the policy decision input.
- sandbox profile selection: executors MUST run within a policy-selected sandbox profile aligned to consequence tier (isolation, filesystem controls, network egress, device access, and allowed tool classes).
- ephemeral-by-default execution: agent executors SHOULD run on short-lived compute instances (containers/VMs) with immutable images and rapid rotation. Long-lived executors require

explicit justification in policy and tighter monitoring.

- secrets and credentials boundaries: executors MUST NOT hold long-lived credentials. Secrets are brokered per action via the secrets broker; models never receive credential material.
- attestation hooks: where available, runtime posture and attestation identifiers SHOULD be produced and referenced (`attest://...`) so actions and messages can be tied to verified runtime state.

The executor substrate is a bulkhead. It constrains the blast radius of compromised tools, poisoned context, and runaway coordination loops while preserving throughput under contested conditions.

5.7 Model gateway

The model gateway enforces:

- model allowlists by enclave and trust scope
- routing by consequence tier and degraded mode
- budget limits (tokens/compute/time)
- metadata capture for replay and auditing

The model gateway is also the upgrade discipline:

- shadow → canary → promote → rollback
- policy-hash and eval-pack gating

Model routing is not only a performance decision; it is an assurance decision. Minimum additional requirements:

- MAP allowlists: every routed model MUST resolve to an approved Model Assurance Profile (MAP) for the enclave and trust scope. Model routing policies reference MAP hashes, not informal names.
- segmented permissions: permission to use a model for general interactive assistance MUST be separable from permission to use that model in autonomous agent workflows. Agent-building and agent-execution are distinct privileges.
- evidence-grade metadata: model gateway events MUST stamp `modelId`/`modelVersion` (or build hash), MAP hash, route decision, and token/compute consumption so replay can reconstitute the exact reasoning core used.
- assurance-preserving upgrades: model upgrades are promoted only if eval-pack gates pass for the applicable trust scopes. MAP changes (including data handling constraints or hosting boundary changes) are treated as upgrades and MUST be gated and rollbackable.

5.8 Context/Data gateway

This gateway turns "context engineering" into a governed plane:

- tag-aware access controls
- provenance capture (source/time/label/custody pointer)
- minimization and redaction
- freshness SLAs and "context rot" warnings

- production of stable context bundles referenced by action envelopes

5.8.1 Memory governance (memory is data, not prompts)

Agent "memory" is data movement and state mutation and MUST be governed like any other enterprise information flow.

Minimum requirements:

- uniform policy mediation: reads and writes MUST be mediated by the same tag-aware policy surface (ABAC + trust scope + environment posture), regardless of storage type (object, vector, stream, geospatial, media)
- writes are side effects: every write to persistent stores MUST be treated as a governed action (policy decision, budgets, and evidence event)
- provenance and labels: memory writes MUST inherit labels from sources unless policy explicitly redacts or downgrades, and MUST record provenance (for example source context bundle hashes)
- poisoning response: memory poisoning is assumed. The gateway MUST support quarantine, re-index, and selective rollback of compromised knowledge nodes and embeddings

5.9 Tool/Action gateway

This is the most important boundary: it mediates side effects.

- allowlists/denylists by scope and tier
- approvals and quorum requirements for high consequence
- secrets brokerage (tools get secrets; models do not)
- sandboxed execution environments
- idempotency keys and retry control to prevent amplification
- action envelopes emitted to evidence ledger
- tool provenance and attestation stamped into every action envelope

Tool transports are not trust boundaries. When a transport protocol is used to connect an agent to tools and resources (for example persistent sessions for tool discovery and invocation), the Tool/Action Gateway MUST treat the transport as a conduit and MUST still enforce policy at the gateway. For provenance, the gateway SHOULD capture transport metadata (server identity, client identity, connection mode) as evidence attributes on each tool invocation so that downstream exports and dashboards can correlate actions to the concrete execution channel.

5.9.1 Tool/skill supply chain governance (registry + provenance tiers)

Agent ecosystems expand by attaching tools, connectors, and "skills." Open marketplaces make that expansion fast—and create a predictable attack surface.

The ACP therefore treats tools and skills as a governed supply chain:

- **Provenance tiers**
  - *Tier A:* first-party tools (Department-owned) with full pipeline attestation
  - *Tier B:* vetted third-party tools (signed + scanned + evaluated + constrained)

  - *Tier C:* untrusted/community tools (denied by default; allowed only in isolated sandboxes and low-tier scopes, if allowed at all)
- **Onboarding pipeline (minimum)**
  - manifest + schema validation
  - dependency analysis + SBOM generation
  - static analysis and policy linting
  - sandbox behavior tests and tool eval packs
  - signing and publishing to a controlled registry
  - runtime attestation (hash/signature match) enforced by the gateway
- **Operational controls**
  - quarantine workflows for tools with anomalous behavior
  - revocation and emergency denylist distribution
  - registry reputation signals (usage history, incident linkage)

The goal is simple: adding tools expands capability **without** expanding unbounded risk.

5.9.2 Computer-use / GUI actuation tools (OS-level and RPA class)

A special class of tools exists where the "tool" is a computer: mouse/keyboard control, screenshots, UI navigation, and OS-level actions. This class is powerful and fragile, and it is high-risk by default.

Policy requirements for computer-use tools:

- run inside isolated desktop environments (VDI/sandbox) with governed network egress
- restrict accessible applications and UI surfaces by scope
- capture structured evidence: screenshots and/or session capture at policy-defined sampling rates
- enforce per-step budgets (clicks/keystrokes/time) and fan-out limits
- treat this class as **T2 by default** unless explicitly lowered by risk assessment and controls
- require veto windows or approvals for irreversible actions (credential changes, external communications, destructive operations)

These constraints turn "computer use" from a hidden capability into a governed actuation surface.

5.10 Inter-Agent Gateway (IAG)

Multi-agent systems only scale safely if agent-to-agent communication is treated like a governed mesh.

The IAG is intentionally **protocol-neutral**. It can front interoperable protocols such as the Agent2Agent (A2A) protocol or the Model Context Protocol (MCP)—or successor protocols that provide similar semantics.

The ACP does not bet on a single wire protocol; it standardizes the policy surface required no matter what carries the message.

Required policy surface (independent of protocol):

- mutual authentication of sender/receiver identities and runtime provenance

- authorization of message types and peer relationships by trust scope + persona + environment
- schema validation (typed envelopes; no arbitrary prompt blobs as transport)
- rate limits + TTLs + fan-out caps to prevent cascades
- provenance: message hashes, policy hash, sender/receiver ids
- circuit breakers: cascade detection and automatic throttling/quarantine triggers
- evidence emission: ensemble graph metadata sufficient for replay and triage

Agents joining a swarm MUST verify the `trustScopeRef` and `policyBundleRef` hashes before accepting tasks or messages. Unknown or unverified policy artifacts result in refusal, quarantine, or escalation per policy.

5.10.1 A2A <-> MCP crosswalk (how both map to ACP boundaries)

A2A and MCP address different interoperability surfaces. The ACP governs both by applying the same policy primitives—identity, trust scope, budgets, and evidence—at the appropriate gateway.

Interop surface	Typical protocol family	ACP enforcement boundary	Governance focus
Agent <-> Agent (delegation, coordination, swarm messaging)	A2A or equivalent	Inter-Agent Gateway (IAG)	peer allowlists, message-type schemas, fan-out limits, cascade breakers, message provenance, ensemble graph evidence
Agent <-> Tool/Data connectors (capabilities, resources, data access)	MCP or equivalent	Tool/Action Gateway + Context/Data Gateway	tool allowlists, connector provenance tiers, consent/permissions, sampling controls, connector attestation, context provenance
Tool/Connector requests model sampling	MCP sampling flows or equivalents	Model Gateway (with policy hooks)	model selection control, budget enforcement, evidence capture, deny-by-default for server-initiated sampling when prohibited

Interop surface	Typical protocol family	ACP enforcement boundary	Governance focus
Agent output becomes executable action	N/A	Tool/Action Gateway	approvals/quorums, sandboxing, secrets brokerage, action envelopes, rollback/replay

The design goal is not to predict which protocol dominates. The goal is to ensure that whichever protocols are used, they terminate at governed boundaries with consistent controls.

5.10.2 Standards touchpoints (non-normative)

ACP is compatible with multiple standards-based identity and authorization approaches. Implementations MAY select one or more of the following patterns:

- OAuth 2.x and OpenID Connect for delegated access and session-bound "on behalf of" semantics
- workload identity and attestation frameworks (for example SPIFFE/SPIRE) for executor and tool runtime identities
- SCIM for provisioning, lifecycle management, and deprovisioning of agent and tool identities where enterprise IAM systems require standard interfaces
- attribute-centric authorization approaches (for example NGAC-style models) for fine-grained ABAC graphs and delegation semantics

Selection of standards does not change ACP's normative requirements: enforcement occurs at gateways using policy decisions, and evidence is signature-covered and replayable.

5.11 Evaluation harness (functional + adversarial) and tool eval packs

The evaluation harness provides:

- baseline functional tests ("golden tasks")
- policy conformance tests (permission boundaries, tool misuse attempts)
- adversarial tests (retrieval injection, poisoning simulations, inter-agent influence scenarios)
- regression gates against last-known-good
- rollback/containment rehearsal checks

5.11.1 Tool contract design (tools as contracts for non-deterministic callers)

Agents call tools differently than deterministic software. Tool APIs must be designed as contracts for a non-deterministic caller:

- typed inputs and outputs (schema-first)
- explicit preconditions and failure modes (deterministic error taxonomy)
- bounded side effects (idempotent operations where possible)

- small, verifiable outputs (avoid mixing commentary with data payloads)
- safe defaults (read-only by default; explicit "apply" actions separated and tiered)
- strict secrets boundaries (no credential material in tool outputs)

5.11.2 Tool eval packs (gating tool onboarding and tool changes)

New tools and tool changes require tool-specific eval packs, including:

- misuse probes (attempted actions out of scope)
- output-injection probes (tool outputs crafted to steer agents)
- retry amplification tests (error storms and idempotency validation)
- performance/latency tests (avoid tool DoS cascades)
- "computer use" class tests (UI ambiguity, evidence capture, step budgets)

5.12 Evidence ledger and replay service

Evidence is a structured event stream supporting:

- attribution: who/what/under which scope/policy
- replay: reconstructing intent → context → decision → action → outcome
- continuous authorization: evidence inside the system boundary

5.12.1 Lineage graph (data → logic → action → artifact)

Replay is only useful if the system can answer "what changed because of what." ACP therefore requires a lineage graph that ties together data, logic, actions, and produced artifacts.

Minimum lineage requirements:

- every action envelope MUST be linkable to:
  - the context bundles it relied on (hashes)
  - the tool identity and version it invoked
  - the model identity/version (or build hash) used for reasoning steps (when applicable)
  - the policy bundle hash and decision record
  - the produced artifact identifiers (content hashes and references)
- lineage MUST be queryable across time for blast-radius analysis:
  - "show all artifacts produced under trustScopeRef X"
  - "show all actions that used toolVersion Y"
  - "show all outputs derived from a given context bundle hash"
  - "show all actions executed with MAP hash Z"

Lineage is a control-plane capability. It enables fast containment, surgical rollback, and defensible audits without relying on informal operator reconstruction.

5.12.2 Evidence interchange formats (derived, non-authoritative)

ACP evidence is recorded canonically as signature-covered action and message envelopes and persisted in the evidence ledger. Implementations MAY additionally produce derived event records in external schemas to support visualization, analytics, training oversight, and integration with existing observability and audit platforms.

Derived records MUST be treated as non-authoritative representations:

- canonical truth: the signature-covered ACP envelope and its evidence root remain the canonical record of what was requested, what was authorized, and what occurred
- required linkage: every derived record MUST include an immutable pointer to its originating ACP envelope(s) (for example `envelopeHash` and `evidenceRootHash`) so any consumer can resolve back to the canonical record
- immutable binding: derived records MUST carry canonical linkage fields as non-optional extensions (at minimum `envelopeHash` and `evidenceRootHash`)
- no bypass: derived exports MUST NOT become the sole source of auditability. If export fails or is suppressed by policy, canonical evidence capture still occurs
- policy-bound export: exporting evidence to external systems is a governed action. The export path MUST enforce labeling, minimization, redaction, and access constraints consistent with the trust scope and policy bundle (see Observability and SOC/CSSP integration)

Non-normative examples of derived schemas include xAPI, common SIEM event shapes, and internal analytics schemas. Derived formats do not change ACP's canonical evidence model.

5.12.3 Swarm-scale evidence: hierarchical aggregation for practical replay

Swarm replay can be prohibitively expensive if every message and trace is retained at full fidelity. ACP uses hierarchical evidence aggregation:

Per-agent evidence (always-on):

- action envelopes
- inter-agent message envelopes (metadata always; payload by tier/trigger)
- context bundle hashes + provenance pointers (payload capture by tier/trigger)
- policy decisions (inputs/outputs + policy hash)
- drift/anomaly signals (summaries)

Ensemble evidence (always-on, lightweight):

- coordination graph metadata (A2A edges by type and time)
- arbitration outcomes (conflicts, quorums, deadlocks/timeouts)
- budget burn-down time series (aggregate and per-role)
- work-unit DAG summaries (dependencies, blocks, cancellations)

Selective enrichment (triggered):

- full message bodies, full context payloads, and detailed planning traces captured on:
  - anomaly thresholds,

- high-consequence actions,
- investigation holds.

This yields three replay tiers:

1) macro replay (graph + budgets + key decisions) for fast SOC/CSSP triage
2) meso replay (selected agents/intervals) for root-cause on a suspected subgraph
3) micro replay (full payloads) for high-consequence or legal/incident requirements

5.13 Observability and SOC/CSSP integration

ACP emits telemetry so operations teams can see:

- allow/deny rates by scope/tool/data tag
- tool-call distributions and anomaly signals
- model routing changes and regression alerts
- work-unit status and stall signals (blocked, deadlocked, retry storms)
- containment events (quarantine/kill/rollback) with evidence pointers

Telemetry and evidence are often more sensitive than the outputs they describe. ACP therefore treats logs, traces, and evidence as governed resources:

- label-bound access: access to telemetry and evidence MUST be controlled by the same labeling and ABAC mechanisms used for data and artifacts. If evidence references labeled context, evidence access MUST not bypass those constraints
- export is an action: exporting telemetry/evidence to external systems (including SIEM/SOAR) MUST be mediated as a governed tool/action with explicit policy, budgets, and evidence of the export itself
- mutation audit: changes to policy bundles, trust scopes, model routes, tool registries, memory policies, and labeling rules MUST generate audit evidence with before/after hashes, actor identity, `delegationHash` (when present), and a reason code. Policy mutation is a first-class event stream, not an administrative footnote

Exporting telemetry supports monitoring and visualization, but it does not replace canonical evidence capture. Auditability and non-repudiation derive from signature-covered envelopes persisted in the evidence ledger; exports are derived products that MUST link back to canonical hashes.

5.13.1 Swarm observability: SOC/CSSP views

- Identity and delegation: actions grouped by principal chain (owner, agent service principal, on-behalf-of principal), including session binding and denied/approved rates.
- Tool utilization: tool invocations by `toolId`/`toolVersion`, by trust scope, and by consequence tier; include egress-denied and policy-blocked events.
- Data and memory operations: read/write activity by label, by store type (working, episodic, semantic, procedural), including quarantine and rollback events.
- Model usage: `modelId`/`modelVersion` and assurance profile (when used), token/latency/cost budgets, and anomaly triggers.

- Policy mutation: before/after hashes, approvers, and effective-time changes for policy bundles, trust scopes, tool registries, and model routes.
- Trace-to-version replay: drill-down from an output artifact to the exact context hashes, tool versions, model identifiers, and policy decision records that produced it.

Dashboards are consumers of evidence, not sources of authority. Any dashboard view MUST be reconstructible from the evidence ledger.

5.14 Containment and revocation

Containment operates at multiple levels:

- **agent kill:** stop runtime, revoke credentials, invalidate scopes
- **agent quarantine:** keep runtime alive but tool-isolated for triage
- **ensemble freeze:** pause coordination, preserve state for replay
- **ensemble degrade:** force safe mode (local models only; read-only tools)
- **policy lockdown:** tighten scopes rapidly across the ensemble
- **work-unit freeze:** pause one work unit while allowing others to continue

Containment must be fast (seconds), attributable, logged, and—when safe—reversible.

5.15 Resource governance (budget engine)

Budgets are policy-enforced constraints:

- compute (GPU seconds, CPU time)
- tokens and inference cost
- tool calls by category
- data egress/bandwidth
- time
- power (watt-hours) in edge deployments
- risk budget (number of high-consequence actions per window)
- unified token accounting: token usage MUST be attributable by work unit, trust scope, principal chain (`delegationHash`), and model MAP; budgets can be enforced and trended at each level

Budget allocation can be static by tier or dynamic by mission priority (see "Patterns").

5.16 Federation and cross-domain transfer

ACP supports federation by treating identity, scopes, context, and evidence as portable artifacts:

- identity federation aligned to ICAM patterns
- scope translation gates across enclaves (intersection + local caveats)
- cross-domain context transfer as sanitized context bundles (hashes/pointers when payload transfer is forbidden)
- evidence bridging: prove linkage without leaking content

6 Control loops (how the system behaves)

6.1 Action loop: intent → policy → mediated action → evidence

Narrative flow:

1. A work unit is created and bound to a trust scope and budget allocation.
2. An agent forms an intent ("open PR," "update config," "provision account").
3. The agent requests action via the Tool/Action Gateway (not direct tool access).
4. The gateway calls the Policy Engine (PDP) with scope + work-unit constraints + attributes + environment signals.
5. The policy decision returns allow/deny + required approvals + budget impacts.
6. Execution runs in a sandbox with brokered secrets and governed egress.
7. The outcome is sealed as an action envelope and written to the evidence ledger.

6.2 Governance loop: artifacts → tests → promotion → enforcement

ACP governance is GitOps-oriented:

- trust scopes, tool catalogs, interop policies, model routes, and eval packs are versioned artifacts
- CI validates schemas, runs evals, runs adversarial tests
- signed bundles are promoted to enforcement points
- telemetry and evidence feed continuous monitoring and after-action improvement

---

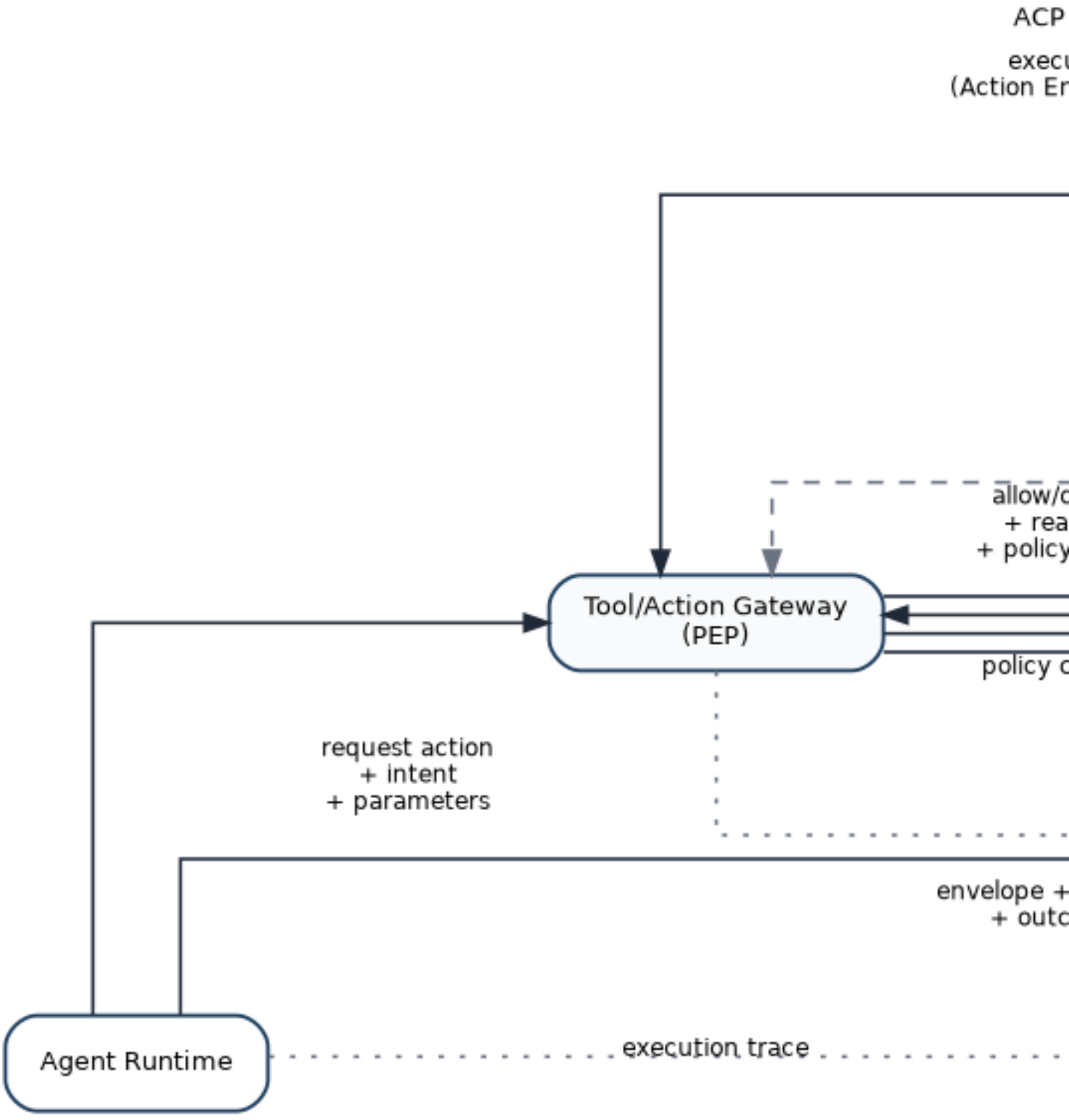
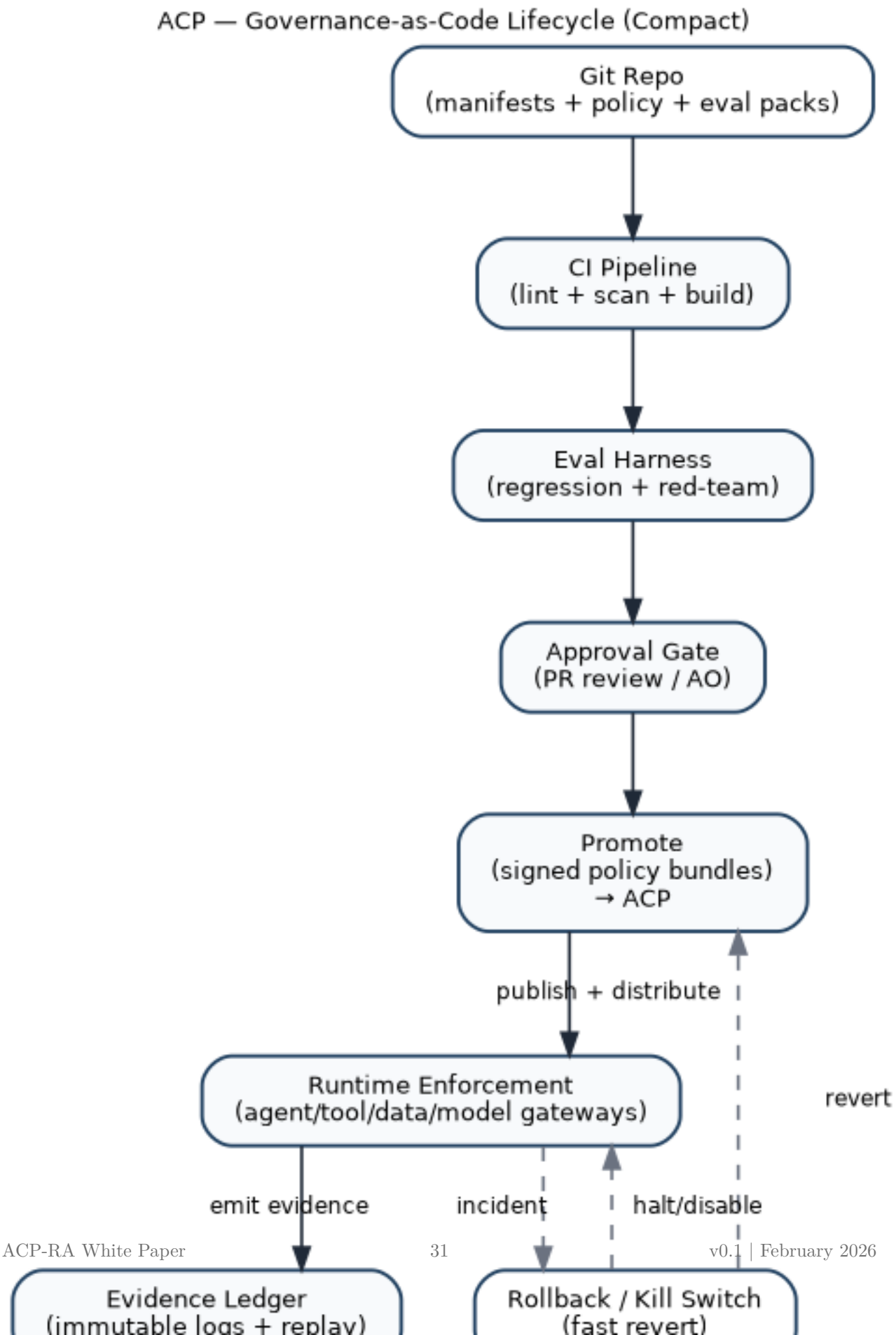


Figure 2: Action Loop


ACP — Governance-as-Code Lifecycle (Compact)
Git Repo
(manifests + policy + eval packs)
CI Pipeline
(lint + scan + build)
Eval Harness
(regression + red-team)
Approval Gate
(PR review / AO)
Promote
(signed policy bundles)
→ ACP
publish + distribute
Runtime Enforcement
(agent/tool/data/model gateways)
revert
emit evidence
incident
halt/disable
Evidence Ledger
(immutable logs + replay)
Rollback / Kill Switch
(fast revert)

7 Multi-agent governance (ensembles and swarms)

Multi-agent behavior is where risk and value both amplify. Without explicit ensemble governance, emergent behavior becomes an incident generator: cascading retries, deadlocks, adversarial influence via compromised peers, and "collective hallucination" reinforced through shared memory.

ACP treats ensembles as first-class objects.

7.1 Coordination patterns (policy-selectable)

Common patterns are supported as declared coordination policies:

- **Hierarchical (orchestrator → workers):** orchestrator decomposes tasks, workers execute narrow scopes. Strong accountability and containment.
- **Peer-to-peer:** distributed planning with explicit arbitration and shared-state controls.
- **Market/auction scheduling:** useful when missions compete for scarce compute/power/bandwidth; requires anti-gaming and auditability.
- **Leader election/rotating coordinator:** avoids single points of failure; requires signed leases and fast failover.

7.2 Trust scope composition and delegation

- Worker scopes are strict subsets of orchestrator scope (intersection).
- Orchestrators delegate budgets as allowances; allowances are reclaimable and time-bounded.
- Ensembles inherit the maximum consequence tier they are capable of initiating unless explicitly forbidden by contract.

7.3 Shared state governance

Shared state is governed by:

- tag-based access controls and minimization rules
- concurrency semantics (leases, idempotency, OCC)
- replayability (state changes reference evidence)
- "memory hygiene" policies (retention, poisoning mitigation, periodic pruning)

Shared state is also shared memory. To prevent "collective hallucination" becoming durable enterprise state, ACP requires:

- memory typing: shared state stores MUST declare what type(s) of memory they contain (for example transient vs persistent; episodic vs semantic) or explicitly partition them
- write constraints: writes MUST be scoped (by work unit, trust scope, and tags) and lease-controlled; long-lived shared writes require tighter tiers or explicit approvals
- provenance-on-write: every mutation MUST reference evidence (policy decision, source context bundle hashes, and actor `delegationHash` when present)
- hygiene as policy: retention windows, pruning cadence, and re-validation triggers MUST be declared in policy bundles and enforced automatically (not left to operator convention)

7.4 Arbitration and deadlock prevention

Ensembles declare:

- conflict policies (who wins, quorum rules, tie-breakers)
- TTLs for tasks/messages
- backoff and retry semantics
- escalation rules for unresolved conflicts

Deadlocks and retry storms are both reliability incidents and security risks.

7.5 Swarm budgets and dynamic allocation

Budgets exist at ensemble and member levels:

- ensemble aggregate budgets bound overall effect
- per-member budgets prevent a single agent from exhausting resources
- dynamic reallocation supports mission priorities under constraint

Allocation mechanisms can include:

- priority queues (mission tiered)
- budget auctions for scarce resources
- throttling policies during degraded modes

7.6 Swarm containment without collapse

Containment must be able to isolate one misbehaving member without collapsing collective behavior:

- quarantine one agent (tool isolation) while the ensemble continues
- freeze coordination while preserving state for replay
- degrade ensemble mode (local models only; read-only tools) when attack indicators rise

---

8 Adversarial robustness and contested/degraded operation

ACP assumes sophisticated adversaries and treats "benign" inputs as untrusted until proven otherwise.

8.1 Threat classes

- context and data poisoning (including retrieval hijacking and shared-memory poisoning)
- tool output deception and supply chain compromise
- model extraction/inversion via repeated calls
- goal drift and reward hacking (where feedback loops exist)
- control plane subversion (policy engine, evidence ledger, revocation channels)
- inter-agent adversarial influence (compromised peers or malicious interop payloads)

8.2 Defenses engineered into ACP

- provenance everywhere (context bundles, message hashes, action envelopes)
- sandboxed execution and secrets brokerage
- strict egress control and budget enforcement
- runtime drift detection (behavior distribution shifts vs baselines)
- circuit breakers at the inter-agent gateway and tool gateway
- continuous adversarial eval packs as promotion gates
- distributed enforcement with cached signed policy bundles to survive PDP outages
- tamper-evident evidence with integrity checks and separate control-plane segmentation

8.3 Degraded modes (policy-driven safe behavior)

Degraded operation is a declared policy mode, not a surprise outage.

Examples:

- **Disconnected:** no external model gateway; local inference only; strict tool denylist; increased escalation.
- **Intermittent:** queue actions; delayed approvals; increase evidence capture for later sync.
- **Denied-model:** forced fallback to smaller/local models; reduced autonomy; tightened budgets.
- **Human-handover:** if confidence drops or novelty rises beyond threshold, shift to human decision authority.

Trust scopes declare which degraded modes are allowed and what behaviors change per mode.

---

9 Human oversight, escalation, and Responsible AI alignment

Human involvement is not binary. It is engineered as governed transitions based on confidence, novelty, consequence, and ethical constraints.

9.1 Escalation taxonomy (what triggers humans)

Escalation triggers include:

- **Confidence:** low calibration, conflicting sources, high uncertainty in plan selection
- **Novelty:** new tool, new data domain, new environment/enclave, unusual dependencies
- **Consequence:** irreversible actions, high-impact changes, public/external communications
- **Ethical/policy flags:** restricted categories, high-impact decision domains, questionable provenance
- **Behavioral anomalies:** drift signals, cascade patterns, tool-use distribution shifts

Trust scopes declare which triggers are binding and what approvals/quorums apply.

9.2 Override surfaces are governed

Human override is a privileged act:

- role-limited, time-bounded, and scoped to specific actions
- dual-control for high-consequence overrides
- recorded as evidence with justification
- auditable and replayable

9.3 Hybrid teaming patterns

ACP supports multiple teaming modes by tier:

- agent proposes, human decides
- agent executes with human veto window
- agent executes under policy with post-hoc review (low-tier only)

9.4 Responsible AI alignment (operationalized as controls)

DoW AI ethical principles are implemented as control-plane behaviors:

- **Responsible:** attributable ownership; governed overrides; audit trails.
- **Equitable:** eval packs include bias/disparate-impact checks where relevant; drift monitoring watches for performance skews.
- **Traceable:** context bundles, action envelopes, and policy decisions enable replay; provenance is mandatory for higher tiers.
- **Reliable:** regression gates and continuous monitoring enforce stability; degraded-mode policies avoid brittle failure.
- **Governable:** kill/quarantine/rollback are first-class; scopes can be tightened centrally and enforced at distributed PEPs.

After-action review loops feed back into trust scope refinement, eval pack expansion, and policy updates.

---

10 Composability, federation, and cross-enclave interoperability

ACP is designed to operate in federated DoW reality.

10.1 Federation patterns

- identity federation aligned to ICAM patterns for mission partners and NPEs
- portable trust scopes as signed artifacts with issuer claims and validity windows
- translation gates when importing scopes into a new enclave:
  - intersect capabilities (never union)
  - map vocabulary and tool catalogs to local enforcement points
  - attach local caveats and retention requirements
  - record translation as evidence

10.2 Cross-domain context transfer

Cross-domain handoffs use context bundles:

- sanitize payloads per policy
- export hashes/pointers when payload transfer is forbidden
- maintain evidence linkage across domains without leaking content

10.3 Portability across clouds, on-prem, and edge

ACP achieves portability by standardizing:

- artifact schemas (scope/policy/interop/eval/evidence/work-units)
- gateway behaviors (tool, model, context, inter-agent)
- evidence event formats

This allows consistent enforcement whether workloads run behind CNAP patterns, on K8s factories, or on disconnected edge nodes.

---

11 Lifecycle of the ACP itself

The ACP will evolve at the same tempo it enables. Control plane upgrades must not break enforcement.

11.1 Safe upgrades and migrations

- policy bundles are versioned and signed; PEPs support dual policy versions during transitions
- trust scope schema versioning with migration tooling and CI validation
- tool/skill registry schema migrations preserve provenance and scanner evidence
- rollback by switching bundle pointers to last-known-good
- compatibility tests and adversarial eval packs required for ACP changes

11.2 Load, chaos, and attack simulation

ACP is tested like a mission system:

- load tests with large ensembles, high A2A chatter, and many concurrent work units
- chaos experiments (PDP outage, model gateway denial, degraded network)
- red-team simulations targeting policy, ledger, registry, and revocation channels
- replay drills: reconstruct ensemble failures using macro→meso→micro evidence tiers

---

12 Patterns (reusable implementation guidance)

12.1 Pattern: "Work units as the unit of supervision"

Humans supervise work units, not tool calls:

- each work unit is bounded by scope and budgets
- outputs are reviewed at artifact/diff level
- escalation triggers bring humans into the loop at the right time

12.2 Pattern: "Policy-centered mesh"

Central policy decisions, distributed enforcement at every boundary (tool, context, model, inter-agent, work-unit transitions).

12.3 Pattern: "Bulkhead gateways"

Gateways as bulkheads: tool sandboxing, context controls, inter-agent circuit breakers, and supply-chain attestation.

12.4 Pattern: "Budgeted autonomy"

Budgets treated as policy and enforced at runtime; budgets drive safe degradation and allocation under scarcity.

12.5 Pattern: "Tool supply chain governance"

Tools and skills are onboarded and operated like software supply chains: signing, scanning, eval packs, attestation, and quarantine.

12.6 Pattern: "Evidence-first releases"

No promotion without evidence: eval packs, policy hashes, scope signatures, and replayability proofs.

12.7 Pattern: "Ensemble contract"

Multi-agent deployments ship with an ensemble contract defining orchestration, arbitration, shared state, budgets, and containment.

---

13 Metrics, success criteria, and a transition roadmap

13.1 Success metrics

Tempo and delivery

- time from scope/policy PR to deployable signed bundle
- work-unit completion time distributions by tier and mission thread
- model/tool update cycle time (shadow → canary → promote → rollback)
- mean time to quarantine/revoke (MTTQ / MTTR-Q)

Safety and governance

- policy violation attempts per tool/data category
- approval compliance rate for high-tier actions
- evidence completeness rate (required fields present per action)

- replay success rate (can reconstruct runs to acceptable fidelity)

Swarm reliability

- deadlock/livelock frequency
- cascade containment time (detect → throttle → isolate)
- ensemble success rate on golden scenarios

Tool supply chain

- % of tool executions with verified signatures/attestation
- time-to-revoke malicious or unstable tools
- incidence rates by provenance tier (Tier A/B/C)

Adversarial robustness

- red-team pass rate by attack class
- drift detection sensitivity/precision
- anomaly response time (detect → contain)

Resources

- compute and power per unit effect (task completion per watt-hour)
- egress per mission outcome
- budget overrun frequency and root causes

13.2 Maturity levels (ACP-conformance)

- **Level 0:** copilot sprawl; no mediated tools; ad-hoc prompts; minimal evidence
- **Level 1:** identified agents (NPE), basic logging, manual controls
- **Level 2:** governed actions (trust scopes + tool gateway + evidence ledger)
- **Level 3:** supervised autonomy (work units + eval gates + drift monitoring + rehearsed rollback/containment)
- **Level 4:** swarm governance (ensemble contracts + inter-agent gateway + arbitration + swarm budgets + swarm dashboards)
- **Level 5:** federated & contested (cross-enclave scope translation + degraded modes + control-plane survivability drills + registry hardening)

13.3 Transition roadmap (copilots → governed agents → supervised autonomy → governed swarms)

1. **Instrument first:** standard evidence schema; onboard agents as NPEs.
2. **Mediate doing:** enforce tool gateway; budgets; sandboxing; secrets brokerage; registry onboarding for tools.
3. **Codify scopes:** require trust scope manifests; signatures; policy bundle enforcement.
4. **Introduce work units:** supervise long-running tasks; bind outputs to work_unit_id; integrate approvals and evidence drill-down.
5. **Gate releases:** eval packs as promotion gates; canary/rollback for tools/models/policies.

6. **Scale to ensembles:** ensemble contracts; inter-agent gateway; arbitration rules; swarm observability.
7. **Federate:** scope translation; cross-domain context/evidence bridging; portability across environments.

---

14 Recommendations

- Treat every agent as a non-person entity with enterprise identity attributes and lifecycle controls.
- Require a signed trust scope manifest for every agent and ensemble, and compose authority by intersection.
- Make work units the unit of supervision: budgets, dependencies, checkpoints, and evidence roots.
- Mediate all side effects through gateways (tool, context, model, inter-agent) with policy enforcement and audit.
- Gate promotions with eval packs and adversarial tests; require rollback and containment rehearsal.

15 Conclusion

ACP-RA treats autonomy as an engineered control system. It separates planning from doing, centralizes policy decisions while distributing enforcement, and makes evidence and containment first-class. This enables faster iteration without turning speed into unmanaged risk, including for multi-agent ensembles operating in contested and degraded environments.

---

16 Appendix: Minimal artifact set (GitOps-ready)

At minimum, version-control the following:

- `trust-scope/*.yaml`
- `ensemble/*.yaml`
- `work-units/*.yaml` (or work-unit schemas and templates)
- `tool-catalog.yaml`
- `tool-registry/*.yaml` (tool manifests, provenance tier, signatures, SBOM pointers)
- `inter-agent-policy.yaml`
- `connectors/*.yaml` (MCP or equivalent tool transports: servers/resources/prompts allowlists, if used)
- `model-routing.yaml`
- `model-assurance/*.yaml` (Model Assurance Profiles; hosting + data handling + eval references)
- `sandbox-profiles/*.yaml` (executor and tool sandbox profiles by tier and enclave)
- `lineage-schema/*.json` (required lineage fields and query keys)
- `delegation-schema/*.json` (delegation chain fields and signature coverage requirements)

- `memory-policies/*.yaml` (retention, write rules, poisoning mitigation, pruning cadence)
- `policy-trust-anchors/*.yaml` (accepted signing authorities and hash allowlists for policy bundles and trust scopes)
- `context-sources.yaml`
- `eval-packs/*.yaml`
- `tool-evals/*.yaml`
- `evidence-schema/*.json`
- `evidence-export-profiles/*.yaml` (REQUIRED if exporting derived evidence: minimization, linkage requirements, and governed sinks)
- `dashboards/*.json` (SIEM/SOAR/SOC views)
- `dashboard-queries/*.yaml` (optional: saved lineage/governance queries derived from evidence)
- `runbooks/*.md` (containment, rollback, degraded-mode transitions)

---

17 Appendix: Example ensemble contract (illustrative)

```yaml
apiVersion: acp.dod/v1
kind: Ensemble
metadata:
  name: ops-planning-ensemble-alpha
spec:
  consequenceTier: T2
  orchestratorRef: "npe:agent/orchestrator-77c9"
  members:
    - ref: "npe:agent/geo-analyst-112a"
      role: analysis
    - ref: "npe:agent/logistics-55bd"
      role: planning
    - ref: "npe:agent/policy-checker-9ef1"
      role: safety
  coordination:
    pattern: hierarchical
    arbitration:
      conflictPolicy: orchestrator-final
      quorumRules:
        - actionType: irreversible
          quorum: "2-of-3"
    timeouts:
      taskTTLSeconds: 900
      messageTTLSeconds: 120
  budgets:
    aggregate:
      gpuSeconds: 7200
      toolCallsPerHour: 300
      egressMBPerHour: 50
    perRole:
      analysis:
```

```
        toolCallsPerHour: 80
      safety:
        toolCallsPerHour: 30
  interAgentPolicy:
    requireSignedMessages: true
    allowedMessageTypes:
      - task.assign
      - task.result
      - artifact.share
    rateLimits:
      maxMessagesPerMinutePerMember: 120
      fanOutCaps:
        maxRecipientsPerMessage: 8
  safety:
    quarantineOn:
      - signal: drift.high
      - signal: iag.cascade
  degradedModesAllowed:
    - intermittent
    - denied-model
```

---

18 Appendix: Example work unit template (illustrative)

```
apiVersion: acp.dod/v1
kind: WorkUnit
metadata:
  name: "wu-opsplan-2026-02-10-0007"
spec:
  trustScopeRef: "trustscope://ops-planning/T2@sha256:..."
  policyBundleRef: "policy://bundle/2026-02@sha256:..."
  budgets:
    gpuSeconds: 900
    toolCalls: 50
    egressMB: 10
    wallClockSeconds: 1800
  dependencies:
    requires:
      - "wu-opsplan-2026-02-10-0002"
  checkpoints:
    frequencySeconds: 180
    artifacts:
      - type: "plan"
      - type: "diff"
      - type: "evidence-summary"
  escalation:
    onBlockedSeconds: 300
    onNovelToolUse: true
```

19 Appendix: Example model assurance profile (illustrative, non-normative)

```yaml
apiVersion: acp.dod/v1
kind: ModelAssuranceProfile
metadata:
  name: "map-commercial-no-retention"
spec:
  modelId: "model:frontier-x"
  modelVersion: "sha256:<immutable-build-hash>"
  hosting:
    enclave: "npe-il5"
    region: "us-gov"
  dataHandling:
    promptRetention: "none"
    completionRetention: "none"
    trainingUse: "prohibited"
    telemetryContent: "metadata-only"
  usageModesAllowed:
    - "interactive-assist"
    - "agent-execution"
  evalRequirements:
    evalPackRefs:
      - "eval://pack/agentic-baseline@sha256:..."
      - "eval://pack/adversarial-injection@sha256:..."
  lifecycle:
    canaryPolicy: "shadow-then-canary"
    rollback: "enabled"
```

20 Appendix: Example sandbox profile (illustrative, non-normative)

```yaml
apiVersion: acp.dod/v1
kind: SandboxProfile
metadata:
  name: "sandbox:t2"
spec:
  isolation:
    mode: "container"
    ephemeral: true
  filesystem: "read-only-root"
  network:
    egressPolicyRef: "egress:t2-restricted"
    allowDns: true
```

```
    allowOutboundDomains:
      - "internal.gov.service"
  secrets:
    brokerRef: "secrets://broker/v1"
    allowDirectSecretRead: false
  observability:
    sessionCapture: "enabled"
    evidenceSampling: "high"
  toolClassesAllowed:
    - "data.read"
    - "repo.pr.create"
    - "ticket.update"
```

---

21 Appendix: Alignment to NIST "Software and AI Agent Identity and Authorization" focus areas (draft, as of 2026-02-16)

This appendix maps ACP control surfaces to the identity and authorization considerations currently being explored by NIST NCCoE for software and AI agents. This is alignment context, not a dependency and not a normative requirement source.

1. Identification
   - ACP principals include human subjects and non-person entities (NPEs) and support stable identities (service principals) and ephemeral identities (run/session identifiers) bound to evidence.
   - Trust scopes bind allowed actions to explicit principals and environments.
2. Authentication
   - ACP requires authenticated service identity for agent executors and authenticated human identity for delegated ("on behalf of") execution.
   - Key lifecycle (issuance/rotation/revocation) is an implementation requirement for executor and tool identities.
3. Authorization
   - ACP authorization is evaluated continuously using PDP/PEP enforcement at model, data/context, tool/action, and inter-agent gateways.
   - Least privilege is enforced by trust scope constraints, policy bundles, environment posture, budgets, and consequence tiers.
   - Delegation is explicit, session-bound, and intersected across owner, agent service principal, and on-behalf-of principal when present.
4. Auditing and non-repudiation
   - ACP emits signature-covered envelopes for all governed actions and messages and persists these into an evidence ledger sufficient for replay, attribution, and forensic reconstruction.
   - Derived exports remain non-authoritative and MUST link back to canonical envelope hashes.

5. Prompt injection prevention and mitigation
   - ACP treats retrieved content and tool outputs as data, not authority.
   - Context ingestion and tool outputs are label-scoped, provenance-tracked, and subject to quarantine, rollback, and minimization.

---

22 Appendix: Memory modalities (illustrative)

These modalities are implementation mental models. They do not create authority and do not change ACP's trust boundaries.

- Working memory: the information at the disposal of the agent during the current control loop (prompt inputs, intermediate variables, scratchpads). Working memory is transient by default and MUST NOT be persisted unless policy explicitly allows it.
- Episodic memory: information retained across execution sessions that is primarily temporal (what happened, when, under which scope). Episodic memory SHOULD be implemented as structured, queryable records tied to work units and evidence roots.
- Semantic memory: durable knowledge organized for retrieval (knowledge nodes, vector indexes, curated references). Semantic memory MUST be governed as a knowledge base: controlled ingestion, labeling, provenance, and periodic re-validation.
- Procedural memory: durable instructions and routines that shape behavior (prompt templates, policies-as-code, workflow logic, tool schemas, agent "skills"). Procedural memory MUST be treated as a supply chain artifact: versioned, signed, evaluated, and promoted through gates.

---

23 Appendix: References (authoritative and industry sources)

URLs are listed for traceability; downstream repositories should pin to specific versions/hashes where possible.

23.1 DoW / DoW CIO

- DoW CIO, *Reference Architecture Description* (June 2010): https://dodcio.defense.gov/Portals/0/Documents/Ref_Archi_Description_Final_v1_18Jun10.pdf

- DoW CIO, *DoW Zero Trust Reference Architecture v2.0* (2022): https://dodcio.defense.gov/Portals/0/Documents/Library/%28U%29ZT_RA_v2.0%28U%29_Sep22.pdf

- DoW CIO, *ICAM Federation Framework* (2024): https://dodcio.defense.gov/Portals/0/Documents/Cyber/ICAM-FederationFramework.pdf

- DoW CIO, *Cloud Native Access Point (CNAP) Reference Design v1.0* (2021): https://dodcio.defense.gov/Portals/0/Documents/Library/CNAP_RefDesign_v1.0.pdf

- DoW CIO, *DevSecOps Continuous Authorization Implementation Guide* (2024): https://dodcio.defense.gov/Portals/0/Documents/Library/DoDCIO-ContinuousAuthorizationImplementationGuide.pdf

- DoW CIO, *cATO Evaluation Criteria* (2024): https://dodcio.defense.gov/Portals/0/Documents/Library/cATO-EvaluationCriteria.pdf

- DoW CIO, *Continuous Authorization to Operate (cATO) memo* (2022): https://media.defense.gov/2022/Feb/03/2002932852/-1/-1/0/CONTINUOUS-AUTHORIZATION-TO-OPERATE.PDF

- DoW CIO, *AI Cybersecurity Risk Management Tailoring Guide* (2025): https://dodcio.defense.gov/Portals/0/Documents/Library/AI-CybersecurityRMTailoringGuide.pdf

- DoW, *Implementing Responsible AI in the DoW* (May 2021): https://media.defense.gov/2021/May/27/2002730593/-1/-1/0/IMPLEMENTING-RESPONSIBLE-ARTIFICIAL-INTELLIGENCE-IN-THE-DEPARTMENT-OF-DEFENSE.PDF

- DoW, *Responsible AI Strategy and Implementation Pathway* (June 2022): https://media.defense.gov/2022/Jun/22/2003022604/-1/-1/0/Department-of-Defense-Responsible-Artificial-Intelligence-Strategy-and-Implementation-Pathway.PDF

- DoDD 3000.09, *Autonomy in Weapon Systems* (Jan 2023): https://www.esd.whs.mil/portals/54/documents/dd/issuances/dodd/300009p.pdf

23.2 Industry protocols and agent platform patterns

- OpenAI, *Introducing Codex* (2025): https://openai.com/index/introducing-codex/

- OpenAI, *Introducing the Codex app* (2026): https://openai.com/index/introducing-the-codex-app/

- Anthropic, *Writing effective tools for agents — with agents* (2025): https://www.anthropic.com/engineering/writing-tools-for-agents

- Anthropic Claude Docs, *Computer use tool* (2025/2026): https://platform.claude.com/docs/en/agents-and-tools/tool-use/computer-use-tool

- Google Developers Blog, *A2A: a new era of agent interoperability* (2025): https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interoperability/

- Google Developers Blog, *Google Cloud donates A2A to Linux Foundation* (2025): https://developers.googleblog.com/en/google-cloud-donates-a2a-to-linux-foundation/

- Linux Foundation, *Agent2Agent Protocol Project launch* (2025): https://www.linuxfoundation.org/press/linux-foundation-launches-the-agent2agent-protocol-project-to-enable-secure-intelligent-communication-between-ai-agents

- Model Context Protocol, *Specification (2025-06-18)*: https://modelcontextprotocol.io/specification/2025-06-18

- Model Context Protocol, *Sampling* (2025-06-18): https://modelcontextprotocol.io/specification/2025-06-18/client/sampling

23.3 Tool/skill ecosystem risk signals (supply chain lessons)

- The Verge, *OpenClaw's AI 'skill' extensions are a security nightmare* (2026): https://www.theverge.com/news/874011/openclaw-ai-skill-clawhub-extensions-security-nightmare

- Reuters, *China warns of security risks linked to OpenClaw open-source AI agent* (2026): https://www.reuters.com/world/china/china-warns-security-risks-linked-openclaw-open-source-ai-agent-2026-02-05/

- Cisco, *Personal AI Agents like OpenClaw Are a Security Nightmare* (2026): https://blogs.cisco.com/ai/personal-ai-agents-like-openclaw-are-a-security-nightmare

23.4 Prior papers (conceptual anchors)

- Adam Boas, *From AI Force Multiplication to Force Creation*: https://anboas.github.io/adamboas.info/writing/agentic-force-creation/

- Adam Boas, *From PDFs to Pull Requests (Code-as-Policy)*: https://anboas.github.io/adamboas.info/writing/code-as-policy/